Document 3 — New Developer Setup & Deployment

Everything a new developer needs: set up locally, understand the workflow, and deploy the full production stack from scratch.

Server: langgraph.sudoxp.com (13.203.182.189) · Repo: github.com/Harikarthik01/LangGraph-Development · Model: claude-sonnet-4-6 · Updated 2026-07-07

> This guide takes a **new developer** from zero to productive: install the tools, run the project locally with Studio, learn the daily workflow, and (for whoever maintains the server) **deploy the entire production stack** — Docker + Postgres + Redis + Caddy HTTPS — from scratch.

§0 Contents

§A Local developer setup

Prerequisites: Python 3.11+ (or 3.12), Git, a free LangSmith account, and access to the shared API keys (Anthropic + LangSmith).

1. **Clone the repo**

```
git clone https://github.com/Harikarthik01/LangGraph-Development.git LangGraph
cd LangGraph
```

2. **Create a virtual environment & install**

```
python3.11 -m venv .venv
source .venv/bin/activate
pip install -U pip
pip install -U "langgraph-cli[inmem]" -r requirements.txt langchain-anthropic
```

3. **Create** `.env` with the shared keys (never committed):

```
ANTHROPIC_API_KEY=sk-ant-...
LANGCHAIN_TRACING_V2=true
LANGCHAIN_ENDPOINT=https://api.smith.langchain.com
LANGCHAIN_API_KEY=lsv2_pt_...
LANGCHAIN_PROJECT=Ai-Poc-Dev
LANGSMITH_API_KEY=lsv2_pt_...
```

4. **Run it locally (opens Studio)**

```
set -a; source .env; set +a
langgraph dev
# Studio opens at smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024
```

> **You're set.** Edit `agent.py`, save, and Studio hot-reloads. Build and test agents entirely on your machine before pushing.

§B The daily workflow

```
1. Edit / create an agent          (agent.py, researcher.py, ...)
2. Test locally                    langgraph dev  → Studio
3. Commit & push                    git add . && git commit -m "..." && git push
4. Deploy on the server            git pull && sudo docker compose up -d --build
```

> **Rule:** build & test **locally** (Studio works on localhost). The **server runs** the finished agents and serves the API. Never edit code directly on the server.

§C Full server deployment from scratch

For setting up a brand-new server (what the live one runs).

1. Create the server (AWS Lightsail)

Setting	Value
Blueprint	OS Only → Ubuntu 22.04 LTS
Plan	$24/mo — 4 GB RAM, 2 vCPU, 80 GB SSD
Static IP	Attach one (e.g. 13.203.182.189)
Firewall (open ports)	22 (SSH), 80 (HTTP), 443 (HTTPS), 8123 (API)

Firewall note: all four ports must be open. **443 is required for HTTPS** — a missing 443 rule is the most common deployment snag.

2. Install Docker

```
sudo apt-get update -y
curl -fsSL https://get.docker.com | sh
sudo docker --version && sudo docker compose version
```

3. Clone the code & add secrets

```
cd ~
git clone https://github.com/Harikarthik01/LangGraph-Development.git LangGraph
cd LangGraph
nano .env        # paste the 6 keys (same as local .env)
```

4. Build & launch the stack

```
sudo docker compose up -d --build       # ~3-5 min first time
sudo docker compose ps                   # expect 3 healthy services
curl http://localhost:8123/ok            # {"ok":true}
```

What starts: `langgraph-api` (the server) + `langgraph-postgres` (memory) + `langgraph-redis` (queue). The stack auto-restarts on crash (`restart: unless-stopped`) and on reboot (Docker starts on boot).

§D HTTPS + domain setup (Caddy)

Gives the server a secure `https://` address so Studio and apps can connect securely.

1. **Point DNS** — in your domain registrar (GoDaddy), add an **A record**: `langgraph → 13.203.182.189` (→ `langgraph.sudoxp.com`). Verify: `dig +short langgraph.sudoxp.com`

2. **Install Caddy**

```
sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https curl
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' | sudo gpg --dearmor -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' | sudo tee /etc/apt/sources.list.d/caddy-stable.list
sudo apt update && sudo apt install -y caddy
```

3. **Configure Caddy** (auto-HTTPS + API-key auth + CORS for Studio). Public metadata endpoints (`/ok` , `/info` , `/docs`) stay open so Studio can validate the connection; everything else (running agents, threads, data) requires `X-Api-Key` :

```
sudo tee /etc/caddy/Caddyfile > /dev/null <<'EOF'
langgraph.sudoxp.com {
    @preflight method OPTIONS
    @needsauth {
        not path /ok /info /docs /openapi.json
        not method OPTIONS
        not header X-Api-Key "YOUR_SECRET_KEY"
    }
    handle @preflight {
        header Access-Control-Allow-Origin "{http.request.header.Origin}"
        header Access-Control-Allow-Methods "GET, POST, PUT, PATCH, DELETE, OPTIONS"
        header Access-Control-Allow-Headers "{http.request.header.Access-Control-Request-Headers}"
```

```
        header Access-Control-Allow-Credentials "true"
        header Access-Control-Max-Age "3600"
        respond 204
    }
    handle {
        route {
            respond @needsauth "Unauthorized" 401
            reverse_proxy localhost:8123 {
                header_down Access-Control-Allow-Origin "{http.request.header.Origin}"
                header_down Access-Control-Allow-Credentials "true"
            }
        }
    }
}
EOF
sudo systemctl reload caddy
```

Generate a key with `openssl rand -hex 24`. Rotate anytime: edit the Caddyfile + `sudo systemctl reload caddy`. The `/info` exemption lets LangGraph Studio's connection probe succeed (it doesn't send the key); credit-spending endpoints stay protected.

4. **Verify** — Caddy auto-fetches a free Let's Encrypt certificate (~30s). Test: `curl https://langgraph.sudoxp.com/ok` → `{"ok":true}`

5. **Connect Studio** — open `smith.langchain.com/studio/?baseUrl=https://langgraph.sudoxp.com`. If it says "domain not allowed," click **Configure connection** → **Advanced Settings** and add `langgraph.sudoxp.com` to the allowed list.

§E Operations — run, update, troubleshoot

Everyday commands

```
sudo docker compose ps                # status of all 3 services
sudo docker compose logs -f langgraph-api   # live logs
sudo docker compose restart           # restart the stack
sudo docker compose down              # stop (keeps data volume)
sudo docker compose up -d             # start again
```

Deploy a code update

```
cd ~/LangGraph
git pull
sudo docker compose up -d --build     # rebuild with new code
# .env is untouched (not in git); your keys stay put
```

Troubleshooting

Problem	Likely cause	Fix
HTTPS times out	Firewall 443 not open	Add HTTPS/443 rule in Lightsail
Studio "domain not allowed"	Domain not in allowlist	Configure connection → add domain
Studio "connection refused" (remote HTTP)	No HTTPS on server	Set up Caddy (Part D)
Build fails: langgraph version clash	App package named "langgraph"	Dockerfile: rename package to agent-app
Container unhealthy	Bad .env / missing key	`docker compose logs langgraph-api`
Agent forgets everything	Not using threads	Use `/threads/{id}/runs/wait`
No traces in LangSmith	Tracing off / wrong project	Check `LANGCHAIN_TRACING_V2=true`
Cert not issued	DNS not pointing to server, or port 80 closed	Check A record + firewall 80

Security checklist

- ☐ Restrict port 8123 to team IPs (or rely on HTTPS + future auth)
- ☐ Never commit `.env` (it's git-ignored — keep it so)
- ☐ Rotate any API key that leaks (Anthropic console / LangSmith settings)
- ☐ HTTPS is on (Caddy) — prefer the domain over the raw IP